Taming the Performance Unpredictability of the Maximum k-Plex Finding Algorithms by Machine Learning: [Experiments & Analysis]

Anonymous Author(s)

Abstract

A k-plex is a dense structure where every vertex can be disconnected with at most k vertices. Finding a maximum k-plex (MkP) in a big graph is important in many real applications such as community detection and biological network analysis. Numerous MkP algorithms have emerged in recent years featuring a broad range of sophisticated pruning techniques that fall in three categories, those for (1) branching, (2) bounding, and (3) reduction during subgraph exploration. Two recent winners are (1) U-MkP which features effective branching methods, and (2) kPEX which features an alternated reduction-and-bound (abbr. AltRB) method, both significantly beating prior works but have not been directly compared.

In this paper, we explore how to combine U-MkP's branching technique with the AltRB method to achieve the best performance. U-MkP adopts a sparse data representation that favors buffer reuse during backtracking, while kPEX adopts a dense bit encoding representation that favors set operations. There is no clear winner of the data representations so we implement both versions, and they can enable U-MkP's branching technique and the AltRB method, giving five algorithm variants. However, the running time of some variants can be orders of magnitude faster than others, but there is no clear winner or loser among these variants; the results vary case by case in terms of the graph and the value of k and is highly unpredictable. We, therefore, explore the use of machine learning to predict the fastest algorithm based on the graph features after an initial reduction pruning as well as the value of k, and extensive experiments show that both XGBoost and Random Forest achieve a prediction accuracy above 93%, which improves to above 99% if we allow 50% time tolerance above the fastest algorithm. With a duel-to-kill strategy where two algorithms duel by running concurrently and the first to complete kills the other, XGBoost + SVM (RBF kernel) can avoid the few critical prediction failures from when using XGBoost alone, achieving ideal algorithm selection. Our source code is released at https://anonymous.4open.science/r/kplex-ML-9655.

ACM Reference Format:
Anonymous Author(s). 2025. Taming the Performance Unpredictability of the Maximum k-Plex Finding Algorithms by Machine Learning: [Experiments & Analysis]. In . ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

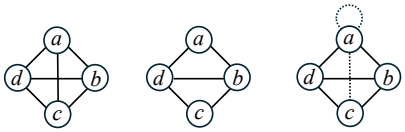


Figure 1: Examples of a Clique and a 2-Plex

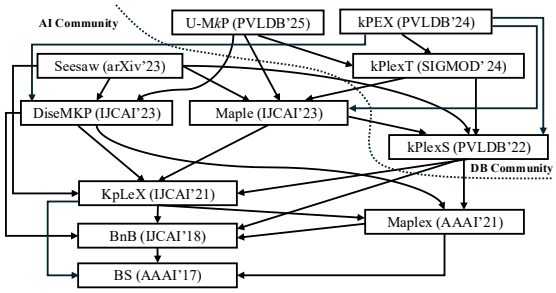


Figure 2: Reported Algorithm Dominance Relationships

1 Introduction

Finding cohesive subgraphs in a large graph is useful in applications such as finding protein complexes or biologically relevant functional groups [5, 8, 17, 25] and social communities [16, 21]. Clique is a classic notion of cohesive subgraph where every pair of vertices are connected by an edge (see Figure 1(a)). However, communities rarely appear in the form of cliques in real graphs due to various reasons such as the existence of data noise [12, 13, 22, 27]. As a relaxed clique model, k-plex [24] is defined as a graph where every vertex v is adjacent to all but at most k vertices (including v itself). Figure 1(b) shows a 2-plex where every vertex is not adjacent to at most $k = 2$ vertices. For example, a is not connected to $\{a, c\}$ (see dotted edges in Figure 1(c)). It has found extensive applications in the analysis of social networks [24], especially in the community detection [13, 22]. However, mining k-plex structures is NP-hard [6, 20], so existing algorithms rely on branch-and-bound search which runs in exponential time, but utilize effective pruning techniques to make the search process tractable on large graphs.

In this paper, we study the problem of finding a maximum k-plex (MkP), i.e., only one is returned when there are ties. An MkP algorithm can be extended to find all MkPs using the method introduced in [4]. In recent years, there is a surge of MkP algorithms with new pruning techniques proposed by the DB and AI communities to significantly speed up computation, such as U-MkP [4], kPEX [15], kPlexT [10] Maple [26], Seesaw [29], DiseMKP [18], kPlexS [9], KpLeX [19], Maplex [30], BnB [14] and BS [28]. Figure 2 shows the performance dominance relationships reported by the respective papers to their compared algorithms. Among them, U-MkP and kPEX are winners that significantly beat the other MkP algorithms, but they have not been directly compared with each other yet.

Specifically, U-MkP [4] is a framework that unifies the pruning techniques from all the other MkP algorithms except for kPEX [15]. U-MkP summarizes the techniques into three categories, those for (1) branching, (2) upper bounding, and (3) reduction during subgraph exploration. Through extensive experimental study, U-MkP identifies a default configuration of the pruning techniques that works generally well, featuring effective branching methods that depend on the value of k: S-based branching method (see Section 3.2) is used when $k \leq 5$ and pivot-based method is used when $k > 5$. kPEX [15], on the other hand, features an alternated reduction-and-bound (abbr. AltRB) method that significantly reduces the search space (see Section 3.3). A key goal of this work is to combine U-MkP's branching techniques with the AltRB method to further improve the performance of MkP finding.

Interestingly, our analysis reveals that U-MkP and kPEX employ distinct data structures for maintaining the explored and candidate vertex sets, exhibiting unique tradeoffs between the *ability of incremental updates* and the *computational overhead of set operations*. It is difficult to find simple rules to predict which data representation wins a priori, as results vary significantly across different problem instances. To harness these complementary strengths, we implement: (1) U-MkP's two branching techniques within kPEX, and (2) the AltRB method within the U-MkP framework. Combined with the original U-MkP and kPEX, this yields five competitive algorithms for MkP finding, providing a portfolio of high-performance candidates to choose from for execution at the runtime.

While certain algorithm variants demonstrate orders-of-magnitude speed improvements over others, no single variant consistently outperforms across all scenarios. The relative performance depends intricately on both graph characteristics and the parameter k, exhibiting high unpredictability. To address this, we develop a machine learning approach to predict the fastest algorithm using graph features (after initial reduction pruning) and the value of k.

This prediction is essentially a five-class classification problem, but requires special treatment in the machine learning workflow to optimize algorithm selection. Specifically, in some problem instances, multiple winner candidates demonstrate statistically indistinguishable performance (e.g., $\leq$ 5% faster or simply a tie), requiring a model to accommodate all optimal algorithm options (rather than to force a single prediction) during training and evaluation. Given a problem instance (i.e., a graph plus a parameter k), the focus should be to avoid selecting a poor algorithm option that runs much slower (e.g., by many times) than the fastest.

To curate a dataset for model training and evaluation, we collected a total of 2741 graphs from three sources: SNAP [3], KONECT [1] and Network Repository [2], which include both real and synthetic graphs. As a preprocessing step before subgraph exploration, kPlexS [9] proposes to use a heuristic algorithm to find a large k-plex P as well as an upper bound ub on the size of any MkP in G, by running the peeling algorithm in linear time. If $|P| = ub$, then P is already an MkP and can be directly returned; otherwise (2) we can still prune those search branches that cannot lead to a larger k-plex with size at least $(|P| + 1)$ during subgraph exploration. This preprocessing step is adopted by the latest algorithms including U-MkP [4] and kPEX [15], and our work also adopts it. Among the 2741 graphs, there are 1635 graphs where the fast preprocessing directly finds an MkP, 791 graphs where all five candidate algorithms run for more than 1800 seconds (which is our timeout threshold to avoid long-running jobs) for all the tested values of k, and 198 graphs which entered the subgraph exploration stage but the result is computed in less than a second. After excluding these graphs where finding the fastest algorithm is not that important, we have only 117 graphs remaining in our curated dataset.

Each (dataset, k) combination gives one sample for machine learning, so our dataset is not large and thus we only consider conventional statistical machine learning models and small neural network models. We remark that our subgraph feature extraction does not incur extra cost since all our graph statistics are straightforward to obtain, including the sizes of k-cores which is a byproduct of the preprocessing step that runs the peeling algorithm. Moreover, deploying a trained model for algorithm selection (i.e., inference) given a new graph takes negligible time since we use small models.

For training, we consider a *time-discrepancy-based sample weighting scheme*, which gives a training sample $\mathbf{x}$ a higher weight $w(\mathbf{x})$ if the running time of different algorithm candidates varies a lot. Note that making the correct prediction for such a sample is crucial since a wrong prediction would lead to a much longer running time, while in contrast, the algorithm selection is not important if all candidates run for more or less the same time. We also consider a *tie-aware weight splitting approach*, where given a training sample $\mathbf{x}$, if k algorithms are all tied to be the fastest (we can allow a tolerance, e.g., an algorithm 10% slower than the fastest one is also considered a tie), then we create k instances of this sample with each instance replicating $\mathbf{x}$'s input features but using a different algorithm label from the k algorithms. Each instance has weight $w(\mathbf{x})/k$ so that the weights of all instances sum up to be $w(\mathbf{x})$. We also explored an alternative approach that uses *(dense) probabilistic categorical cross-entropy loss* for supervision, where "soft" probabilistic labels are used so that algorithms with close running times will have similar probabilities of being selected; but it does not work out.

For evaluation, we generalize metrics such as accuracy, precision and recall to accommodate ties, i.e., a prediction is considered correct iff any of the fastest algorithms is selected (again, a small time-difference tolerance is allowed when defining ties). For each model, we also allow instances with prediction failures to be printed in a failure report for users to intuitively examine them and evaluate the severity of each failure, and the reports can be joined for evaluating the effectiveness of a "duel-to-kill" execution strategy.

The main contributions of this paper are summarized as follows:

- We explore the two winner MkP algorithms U-MkP [4] and kPEX [15], and propose a unified algorithm framework to combine their pruning techniques. Using this algorithm framework, we screened thousands of public graphs to curate a dataset of graphs interesting for our learning problem.
- We survey the difference of data structures for vertex sets used by U-MkP and kPEX during subgraph exploration, including their pros and cons, and propose five competitive algorithm candidates decided by the data structure and adopted pruning techniques for learning-based selection.
- For machine learning, we propose techniques to put larger weights on samples where the selection decision is critical during training, and techniques to accommodate winning algorithms with ties during both training and evaluation.

- Extensive experiments are conducted that identify XGBoost and Random Forest to be the best models with an accuracy above 93%, which improves above 99.3% if we allow 50% time tolerance above the fastest algorithm. With a duel-to-kill strategy where two algorithms duel by running concurrently and the first to complete kills the other, XGBoost + SVM (RBF kernel) can avoid the few critical prediction failures from when using XGBoost alone.

The rest of this paper is organized as follows. Section 2 reviews the set enumeration procedure used by MkP algorithms, and surveys recent advances in MkP techniques. Then, Section 3 presents our unified MkP framework, detailing search-space pruning strategies unique to U-MkP and kPEX. Section 4 describes the distinct set representation methods employed by U-MkP and kPEX. Section 5 reports experiments on learning-based selection among our five competitive algorithms with different combination of pruning techniques and set representations. Section 6 concludes this paper.

2 Preliminaries and Related Work

Notations. We consider an undirected and unweighted simple graph $G = (V, E)$, where V is the vertex set, and E is edge set. The degree of a vertex v is denoted by $d_G(v) = |N_G(v)|$. We also define the concept of *non-neighbor*: a vertex u is a non-neighbor of v in G if $(u, v) \notin E$. Accordingly, the set of non-neighbors of v is denoted by $\overline{N_G}(v) = V - N_G(v)$, and we denote its cardinality by $\overline{d_G}(v) = |\overline{N_G}(v)|$. Given a vertex subset $S \subseteq V$, we denote by $G[S] = (S, E[S])$ the subgraph of G induced by S, where $E[S] = \{(u, v) \in E \mid u, v \in S\}$. We simplify the notation $N_{G[S]}(v)$ to $N_S(v)$, and define other notations such as $N_S(v), \overline{N_S}(v)$ and $d_S(v)$ in a similar manner. For an arbitrary graph g, $V(g)$ and $E(g)$ denote the vertex set and edge set of g, respectively. The diameter of G, denoted by $\Delta(G)$ is the shortest-path distance of the farthest pair of vertices in G, measured by the number of hops.

Problem Definition. We next define k-plex and MkP.

Definition 1. *(k-Plex) A graph g is a k-plex if every vertex $v \in V(g)$ has at least $|V(g)|-k$ neighbors in g, i.e., $d_g(v) \geq |V(g)|-k$. Equivalently, g is a k-plex if every vertex $v \in V(g)$ has at most k non-neighbors in g (including v itself as a non-neighbor), i.e., $\overline{d_g}(v) \leq k$.*

Definition 2. *(*Maximum k-Plex Finding*) Given a graph G, the maximum k-plex (MkP) finding problem finds a largest vertex set $P \subseteq V$ such that the subgraph $G[P]$ induced by P is a k-plex.*

Hereditariness and Diameter of k-Plex. Note that k-plex satisfies the hereditary property which says that: any induced subgraph (denoted by g') of a k-plex g is also a k-plex, since a vertex in g' cannot miss more neighbors than those already missed in g.

Moreover, as proved by [28], the diameter of k-plexes with a reasonably large size (which is usually the case for MkPs) is bounded:

Theorem 1. *For a k-plex P and any integer $c \geq 2$, if $|P| > 2k - c$, then $\Delta(P) \leq c$ [28].*

Most works [9, 26] only consider the case when $c = 2$ by assuming $|P| \geq 2k - 1$, which gives the following corollary:

Theorem 2. *Given a k-plex P, if $|P| \geq 2k - 1$, then $\Delta(P) \leq 2$.*

The assumption $|P| \geq 2k-1$ is reasonable for a maximum k-plex P, since natural communities that our MkP problems aim to discover are connected, and $2k - 1$ is relatively small. For example, when $k = 5$, we only require $|P| \geq 9$. Note that a k-plex with $|P| = 2k - 2$ may be disconnected (e.g., formed by two disjoint $(k - 1)$-cliques).

Algorithm 1: Basic Branch-and-Bound Search

```
1 function BB_basic(S, R, g)    # The basic BB(.) variant
2     if (reduce_and_prune(S, R, g) = true) then return
3     for each v ∈ R do
4         BB_basic(S ∪ {v}, R − {v}, g)
5         R ← R − {v}
```

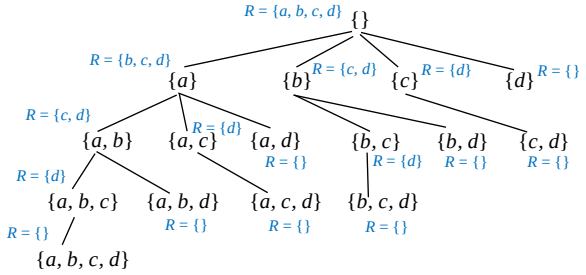


Figure 3: Set-Enumeration Search Tree

Subgraph Exploration by Branch and Bound. All existing MkP algorithms follow (but are variants of) the branch-and-bound search framework shown in Algorithm 1, where S is the set of vertices already included into the current subgraph, and R is the set of candidate vertices yet to be added to S to form a larger subgraph that can become an MkP. Specifically, given a vertex-set pair $\langle S, R\rangle$ in graph g, Line 2 first calls a function reduce_and_prune(S, R, g) to apply reduction and upper-bounding rules, which may decide that the entire search branch to extend S is unpromising so that *true* is returned, in which case Line 2 returns directly to terminate the extension of S. We will discuss this function in more detail in Section 3. If the branch is not pruned, Line 3 then takes the next candidate $v \in R$, and split the search space into two cases: (1) v is in the MkP to find, in which case Line 4 further extends $\langle S \cup \{v\}, R - \{v\}\rangle$ by recursion; (2) v is not in the MkP to find, in which case Line 5 removes v so that it will not appear in future iterations of the for-loop in Line 3. Figure 3 illustrates the search process of Algorithm 1 on a toy graph with four vertices a, b, c and d, which corresponds to a set-enumeration search tree where each node denotes S and we also annotate its corresponding R near the node (assuming that no node is pruned, and that vertices in R are always ordered with $a < b < c < d$). We can see that the search space is perfectly partitioned without redundancy. Note that calling Algorithm 1 on S basically grows the set-enumeration subtree under node S, which we denote by T_S.

Degeneracy Ordering. The k-core of a graph G is its largest induced subgraph with minimum (vertex) degree k. The degeneracy of G, denoted by $D(G)$, is the largest value of k for which a k-core exists in G. For example, Figure 4 shows the 1-core, 2-core and 3-core of a graph. Specifically, the 2-core contains all yellow and red vertices in the yellow dashed contour, since it is the largest induced subgraph where every vertex has degree ≥ 2, as any green vertex has degree 1. Note that even though vertex A has degree 3, it is not in 3-core since its neighbor B has degree 2 so cannot be in

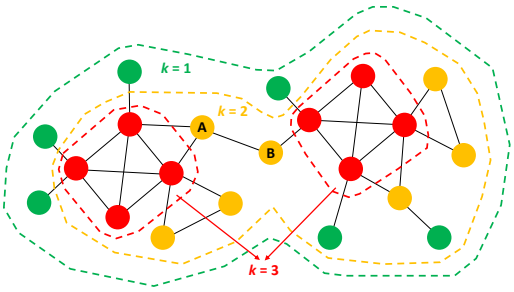


Figure 4: Illustration of k-Cores

Table 1: MkP Algorithms and Their Pruning Techniques [4]

Algorithm	Upper Bounding (UB)					Branching					Reduction						
	S-Based	R-Based	SR-Based	UBR1	UBR2	Binary	Pivot	S-Based	R-Based	SR-Based	Two-Hop	RR1	RR2	RR3	CTCP	BR1	BR2
U-MkP (Our Work)	✓	✓	✓	✓			✓	✓	✓	✓	✓[g]	✓	✓	✓	✓[T]	✓	✓
kPlexT (SIGMOD'24)				✓		✓					✓[g]	✓	✓	✓	✓[T]	✓	✓
Maple (IJCAI'23)				✓			✓				✓[g]	✓	✓	✓	✓[+]	✓	✓
Seesaw (arXiv'23)	✓[+]	✓[+]	✓	—	—	—	—	—	—	—	—	—	—	—	—	—	—
DiseMKP (IJCAI'23)	✓[+]							✓[+]				✓	✓	✓	✓[T]		
kPlexS (PVLDB'22)				✓		✓					✓[g]	✓	✓	✓	✓[+]	✓	✓
KpLeX (IJCAI'21)	✓							✓				✓	✓	✓			
Maplex (AAAI'21)		✓				✓					✓	✓	✓	✓	✓[T]		
BnB (IJCAI'18)						✓						✓	✓	✓			
BS (AAAI'17)							✓				✓	✓	✓	✓			✓

*** Note:** ✓[g]: Two-hop graph constructed; ✓[+]: Improved version; ✓[T]: Top-level only
—: Seesaw only proposes UB rules, no open-source code;

3-core, hence A has at most 2 neighbors in 3-core. The operation of k-core decomposition finds the core number of every $v \in V$, denoted by $core(v)$, which is the largest value of k that v belongs to a k-core. For example, $core(A) = 2$ in Figure 4 since A is in 2-core but not 3-core. The process of k-core decomposition [23] can be computed by a peeling algorithm that repeatedly removes a vertex with the minimum current degree at a time [7]. For the graph G in Figure 4, green vertices will first be removed, followed by yellow ones, and finally the red ones, after which no vertex is remaining so we have $D(G) = 3$. Accordingly, the algorithm also generates the degeneracy ordering of vertices in G where green vertices go first, followed by yellow ones and then red ones. Note that $D(G)$ is a small value in real graphs, since a high-degree vertex tends to appear later in the degeneracy ordering, so many of its neighbors have already been removed by peeling.

Related Work on MkP Algorithms. Recall from Figure 2 on Page 1 that many MkP algorithms have been proposed in recent years. Earlier works are mainly from the AI community (since it is a search problem) including Maple [26], Seesaw [29], DiseMKP [18], KpLeX [19], Maplex [30], BnB [14] and BS [28]; while more recent works are from the DB community including U-MkP [4], kPEX [15], kPlexT [10] and kPlexS [9]. These MkP algorithms have proposed many sophisticated pruning techniques during subgraph exploration that fall in three categories, those for (1) branching, (2) bounding, and (3) reduction. The work of U-MkP [4] unifies all techniques into its framework as summarized in Table 1 (directly from [4]) except for new ones proposed by kPEX [15], and please refer to [4] for the detailed statement and proof of each technique. In this paper, we focus on describing those different techniques adopted by U-MkP's default configuration (that is tuned to work well in general) and kPEX that really matter for performance (note that some pruning techniques turn out to have negligible impact on real performance), while skipping the common techniques enabled

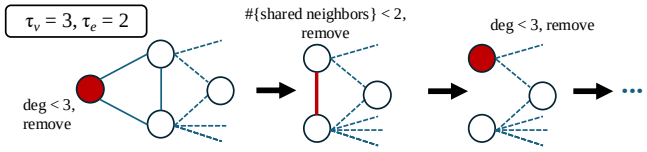


Figure 5: Illustration of CTCP

by all the competitive algorithms, such as UBR2 in Table 1 that is initially proposed by kPlexT [10] and turns out to be so effective that subsequent works U-MkP and kPEX adopt it by default.

To achieve good performance consistently, U-MkP [4] uses S-based branching method when $k \leq 5$ and pivot-based method when $k > 5$. kPEX [15] proposes an alternated reduction-and-bound (abbr. AltRB) method that significantly reduces the search space and is not covered by U-MkP. Besides these effective techniques, U-MkP and kPEX share a very similar algorithm framework making it easy to unify them, except that (1) kPEX replaces CTCP with a variant reduction technique called CF-CTCP, and (2) it replaces the heuristic algorithm originally proposed by kPlexS [9] (to find an initial large k-plex during the preprocessing step) with a different heuristic algorithm called KPHeuris. These changes do not show prominent performance improvement on most graphs as shown in the ablation study results from Tables 12-15 in the appendix of [15].

3 MkP Algorithm Framework and Techniques

In this section, we present the algorithm framework to unify U-MkP [4] and kPEX [15], focusing on the branching methods of U-MkP and the alternated reduction-and-bound method of kPEX.

3.1 A Unified Algorithm Framework

Initial Graph Shrinking & CTCP. Algorithm 2 shows the pseudocode to create top-level search branches $T_{\{v_i\}}$ as adopted by both U-MkP and kPEX, where Line 4 first shrinks the input graph G based on the parameter k. Let P be the current largest k-plex found, then to find a larger k-plex with size at least $|P|+1$, we only need to consider those vertices with degree at least $\tau_v = (|P| + 1 - k)$ (otherwise, the vertex will miss more than k edges in the larger k-plex). Line 4 uses the CTCP (Core-Truss Co-Pruning) reduction method proposed in kPlexS [9] that is a tighter version: we only need to consider those vertices with degree at least $\tau_v = (|P| + 1 - k)$ and those edges (u, v) where u and v share at least $\tau_e = (|P| + 1 - 2k)$ common neighbors (denoted by $d_G(u, v) \geq \tau_e$) [4] . Figure 5 shows an example where vertices and edges below thresholds are alternately pruned. kPEX uses a variant called CF-CTCP that further reduces the redundant computations in CTCP. Since subgraph exploration is conducted on the shrunk G after Line 4, ***our machine learning model uses input features extracted from the shrunk graph rather than the original input graph that is much larger.***

Besides shrinking the initial graph G in Line 4, CTCP is also called in Line 12 (right after Line 11 removes v_i from G), to shrink G further for use by future iterations, since new edges (e.g., v_i's adjacent ones) may have $d_G(u, v)$ reduced below τ_e. By passing $\{v\}$ as the second argument, CTCP can propagate pruning from v and those newly updated edges with $d_G(u, v) < \tau_e$ to minimize its graph update footprint using the method described in [9].

Algorithm 2: Top-Level Branching ($c = 2$)

Input: A graph $G = (V, E)$ and an integer $k \geq 2$
Output: A maximum k-plex in G

```
1  function top_branching(G)
2      P, ub ← find_init(G)    # global variables
3      if |P| ≥ ub then return P
4      CTCP(G, ∅, |P| + 1 − k, |P| + 1 − 2k)
5      Sort [v_1, ..., v_n] following degeneracy ordering of G
6      for i = 1 to |V| do
7          H_1 ← N(v_i) ∩ {v_{i+1}, ..., v_n}
8          H_2 ← N(H_1) ∩ {v_{i+1}, ..., v_n}
9          g ← the subgraph of G induced by ({v_i} ∪ H_1 ∪ H_2)
10         BB({v_i}, V(g) − {v_i}, g)    # call a BB variant
11         Remove v_i from G
12         CTCP(G, {v_i}, |P| + 1 − k, |P| + 1 − 2k)
13     return P
```

Top-Level Branching & Top-Level Subgraphs. In Algorithm 2, we assume there are two globally maintained variables accessible any time during subgraph exploration: (1) P: the current largest k-plex found, and (2) ub: size upper bound of an MkP. These two variables are initialized in Line 2 using a heuristic algorithm that finds a large initial P as a preprocessing step; P will also be updated during subgraph exploration whenever a larger k-plex is found, so that if we know that a branch cannot generate a k-plex larger than $|P|$, it can be pruned without exploration. If $|P|$ equals ub, then the initial P is already an MkP and is thus returned in Line 3. Otherwise, Line 4 then further prunes G using CTCP.

Recall from Theorem 1 that reasonably large k-plexes ($|P| > 2k - c$) have diameter $\Delta(P) \leq c$. In other words, for the top-level nodes v_i ($v_i = a, b, c, d$) in Figure 3, the branch $T_{\{v_i\}}$ under it only needs to consider a subgraph of G, denoted by $g_i = (V_i, E_i)$, where vertices are within c hops from v_i. We call g_i as the ***top-level subgraph*** of $T_{\{v_i\}}$. It is worthwhile to shrink G to g_i, and explore subgraphs in the branch $T_{\{v_i\}}$ over g_i since (1) g_i is much smaller than G so checking the neighbors of each vertex becomes much faster, and (2) this is an efficient one-time processing that can benefit the entire branch $T_{\{v_i\}}$. In contrast, when processing $T_{\{v_i\}}$, even though whenever a vertex v is added to S, we can shrink g_i further to remove vertices more than c hops away from v, this cost is associated with each expansion of S and so not worthwhile.

We then explore branches $T_{\{v_i\}}$ in the degeneracy ordering as shown in Lines 5-6, which keeps the sizes of all g_i's small to avoid having a giant branch that needs to search a deep set-enumeration subtree with potentially high node fanouts. This is because a high-degree vertex v_i tends to appear later in the degeneracy ordering, so V_i has excluded many neighbors of v_i originally in V (see Line 11).

Without loss of generality, assume that $c = 2$, that is, an MkP P has $|P| \geq 2k-1$ (i.e., $|P| > 2k-2$). To create g_i for each v_i, Line 7 first obtains one-hop neighbors of v_i in V_i, and Line 8 then uses them to obtain the two-hop neighbors (where $N(H_1) = \cup_{v \in H_1} N(v)$); finally, Line 9 creates g_i using them (i.e., excluding other vertices in V_i that are > 2 hops away from v_i). Note that it is easy to extend g_i for the general case of c. For example, when $c = 3$, we can additionally compute $H_3 \leftarrow N(H_2) \cap \{v_{i+1}, \ldots, v_n\}$ and then compute g_i to be the subgraph of G induced by $(\{v_i\} \cup H_1 \cup H_2 \cup H_3)$.

Algorithm 3: Reduction and Pruning Function

```
1  function reduce_and_prune(S, R, g)  // returns if T_S is pruned
2      if (|P| = ub) then return true
3      Apply reduction rules to (S, R, g)  // AltRB may prune T_S
4      if g_union = g[S ∪ R] is a k-plex then
5          if |S| + |R| > |P| then P ← S ∪ R
6          return true
7      if partition(S, R, g) = ∅ then return true
8      return false
```

Once g_i is created, Line 10 then explores it by extending $S = \{v_i\}$ using branch and bound. For now, we can think of BB(.) simply as BB_basic(.) presented in Algorithm 1, and a variant will be presented for U-MkP later in Algorithm 4. We then remove v_i from G in Line 11 (similar to Line 5 of Algorithm 1) so that later branches will not consider v_i to avoid redundancy. Note that we do not actually need to do the intersections in Lines 7 and 8 since $\{v_1, \ldots, v_{i-1}\}$ have already been removed in previous iterations.

The Reduction Procedure. Recall from Line 2 of Algorithm 1 that BB(.) first calls reduce_and_prune(.) over the instance (S, R, g) for further reduction before extending S with vertices from R, the pseudocode of which is shown in Algorithm 3. Specifically, Line 1 first checks if the current P already reaches the maximum possible size ub, and if so, it returns *true* to notify BB(.) to terminate its subgraph exploration of T_S (see Line 2 of Algorithm 1). Right after the call of BB(.) in Line 10 of Algorithm 2, we can check if $|P| = ub$ and return P directly if so to terminate early.

Otherwise, we apply reduction rules in Line 3 to reduce the candidate size of R. Note that some rules may directly move some candidates from R into S, such as **BR1** and **BR2** in U-MkP [4]. Also, if the AltRB method of kPEX [15] is applied here, it will return an upper bound of the k-plex size in T_S, denoted by ub_S, so if $ub_S < |P| + 1$, Line 3 can simply return *true* to prune the branch.

Before extending S with vertices in R (e.g., Lines 3–5 in Algorithm 1), we first conduct a look-ahead pruning in Line 4 to see if the entire $g_{\text{union}} = g[S \cup R]$ is a k-plex, and if so, Line 6 returns *true* to prune the branch T_S. Moreover, if $S \cup R$ is larger than the current P, P is updated with $S \cup R$ in Line 5.

Finally, we check if T_S can still produce a k-plex larger than P, and if not, Line 7 returns *true* to prune the branch. The function partition(.) (to be introduced later) uses upper-bound-based pruning to return the subset of vertices in R, denoted by B, that are still worth branching with (i.e., to extend S only with a vertex in B next), and if $B = \emptyset$, there is nothing worth extending with so the entire T_S is pruned. Line 7 is used by U-MkP, but not by kPEX. kPEX already uses AltRB for upper-bound-based branch pruning in Line 3.

3.2 Branching Methods

Refer to Algorithm 1 on Page 3 again, where Line 3 selects a vertex v to branch with at each iteration. In the implementation of kPEX, v is simply selected as the vertex with the smallest degree in $G[S \cup C]$, following the same idea of the degeneracy ordering

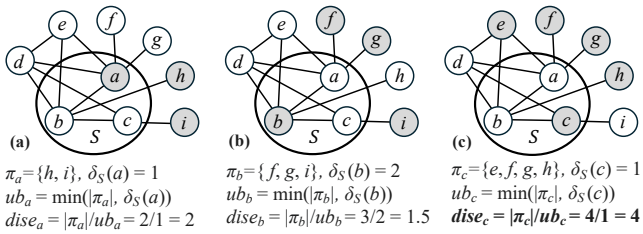


Figure 6: An Example Graph for S-Based Partitioning ($k = 2$)

in top-level branching. However, as [4] has advocated: S-based branching method should be used for small values of k such as when $k \leq 5$, while pivot-based method should be used when k is larger. Therefore, we implement two variants of kPEX that replaces the simple branching method in Lines 3–5 of Algorithm 1 with more advanced methods: (1) **kPEX-S**, which uses S-based branching, and (2) **kPEX-P**, which uses pivot-based branching. We introduce these two branching methods next.

S-Based Upper Bounding. S-based branching is based on S-based upper bounding, which we introduce first. To facilitate discussion, we first define the concept of vertex support.

Definition 3. *(*Support*) The support of a vertex v is defined as $\delta_S(v) = k - |\overline{N_S}(v)|$, which indicates the maximum number of non-neighbors of v that can be added to S.*

Intuitively, $\delta_S(v)$ serves as the non-neighbor "quota" of v that can be added to S (including v itself if $v \notin S$) without breaching the k-plex requirement. We actually maintain $d_S(v)$ incrementally and keep it up to date whenever we move vertices around R and S, so $\delta_S(v) = k - (|S| - d_S(v))$ can always be obtained in $O(1)$ time for use by the pruning techniques. See Appendix J of [4] for the details.

Given $\langle S, R\rangle$, the goal of S-based upper bounding is to compute a size upper bound UB_S on the largest k-plex that S can be extended into using candidates from R (i.e., within branch T_S), by partitioning vertices of R into t subsets $I_1, I_2, \ldots, I_t$. If for each I_i, we can show that at most ub_i vertices from I_i can be added to S without breaching the k-plex requirement, then UB_S can be computed as $|S| + \sum_{i=1}^{t} ub_i$.

S-based upper bounding obtains $\langle I_i, ub_i\rangle$ by partitioning vertices of R based on vertices in S, hence the name *S-based*. Specifically, it partitions R by obtaining from R (1) the common neighbors of all vertices in S as π_0, (2) for each $v_i \in S$, obtain π_i as all the remaining non-neighbors of v_i in R. Consider Figure 6(a) where $S = \{a, b, c\}$ and vertices d to i belong to R: if we check vertices in S by the order $[a, b, c]$, we obtain $\pi_0 = \{d\}$, $\pi_1 = \{h, i\}$, $\pi_2 = \{f, g\}$, $\pi_3 = \{e\}$.

By treating each π_i $(i \geq 1)$ as I_i, we have

$$ub_i = \delta_S(v_i), \tag{1}$$

since it is the quota of non-neighbors of v_i that can be pulled into S. Clearly, we can compute UB_S as follows:

$$UB_S = |S| + |\pi_0| + \sum_{i=1}^{|S|} \min\{|\pi_i|, \delta_S(v_i)\}. \tag{2}$$

However, the value of UB_S depends on the checking order of vertices in S. To find an ordering that leads to small UB_S, DiseMKP [18] proposes the concept of distribution efficiency (dise):

$$dise(\pi_i) = |\pi_i|/ub_i. \tag{3}$$

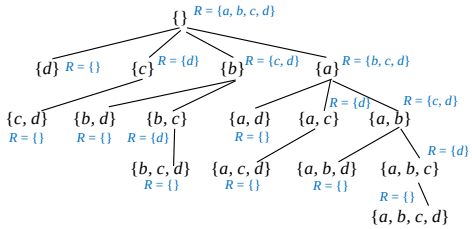


Figure 7: Set-Enumeration Search Tree (Horizontal Flip)

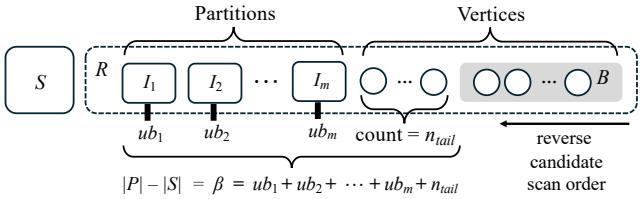


Figure 8: Illustration of Branching Set Computing

Intuitively, we prefer high $dise(\pi_i)$ since we want to remove a large set π_i from R while adding a small ub_i to UB_S. Based on this idea, DiseMKP proposes to greedily check $dise(\pi_i)$ for all the remaining $v_i \in S$ whose π_i has not been selected, and choose the one with the highest $dise(\pi_i)$ as the next I_i. Figure 6 illustrates how I_1 is determined by computing the dise scores, and π_c is picked as I_1 since it has the highest dise. Note that π_i's are initialized as all non-neighbors of v_i that could overlap, so after $I_1 = \pi_c$ is picked, we need to update $\pi_a = \pi_a - I_1 = \{i\}$ and $\pi_b = \pi_b - I_1 = \{i\}$, and then pick I_2 by comparing their dise scores.

Regarding the implementation, we maintain an auxiliary set $\Pi = \cup_{v_i \in S} \pi_i$ that keeps the (potentially overlapping) non-neighbor sets π_i of all $v_i \in S$, which is initialized before computing UB_S, and tracks the candidate partitions to choose next. Note that whenever a partition π_i is chosen, its vertices are removed from all partitions in Π, so if π is previously chosen, it will become empty. The next partition π_j is chosen among those non-empty π_i's in Π, and the detailed algorithm is available in Appendix G of [4].

S-Based Branching. S-based branching utilizes the above partition-based upper bounding technique to allow some branches to be pruned. To see this, consider Figure 7 which shows the horizontal flip of the set-enumeration tree in Figure 3. We can observe that when we scan the candidates from right to left till reaching a candidate $v \in R$, the set-enumeration subtrees rooted at v and all the candidates in front of v in R can only involve vertices in S, v, and those vertices before v in R. For example, for node $S = \{\}$ in Figure 7, when we scan R and reach b, all the three subtrees rooted at nodes $\{d\}$, $\{c\}$ and $\{b\}$ contain vertices in $\{b, c, d\}$. Similarly, for node $S = \{a\}$, when we scan R and reach c, the two subtrees rooted at nodes $\{a, d\}$ and $\{a, c\}$ contain vertices in $\{a, c, d\}$. Now let us consider Figure 8 with $\langle S, R\rangle$, and the last few vertices constitute a set B. Following the above discussion, we know that subtrees rooted at the vertices in $R' = R - B$ can only involve those vertices in $S \cup R'$. If we can show that the size upper bound of a k-plex obtained by extending S with candidates in R', denoted by $UB_S(R')$, cannot exceed $|P|$ (i.e., cannot produce a larger k-plex), then we do not need to extend S with those vertices in R' in the next level. In other words, we only need to branch over vertices in B. We call B the **branching set**. Note, however, that vertices of R' can still appear

Algorithm 4: Partitioning-Based Branch and Bound

Input: Flag σ_{seed}: true if called by the top level.
Flag σ_{up}: a global variable indicating branch pruning

1 {Updates P if a larger maximum k-plex is found}
2 **function** BB_part(S, R, g, σ_{seed})
3 **if** (reduce_and_prune(S, R, g) = *true*) **then return**
4 $B \leftarrow$ partition(S, R, g)
5 $R' \leftarrow R \setminus B$
6 Sort $B = [v_1, \ldots, v_n]$ following degeneracy ordering of g
7 **for** $i = |B|$ *to* 1 **do**
8 **if** (σ_{seed} = *true*) **then** $\sigma_{up} \leftarrow$ *false*
9 **if** (σ_{up} = *true*) **then return**
10 $\tau_{old} \leftarrow |P|$
11 BB_part($S \cup \{v_i\}, R', g,$ *false*)
12 **if** $|P| > \tau_{old}$ **then** $\sigma_{up} \leftarrow$ *true*
13 $R' \leftarrow R' \cup \{v_i\}$

in the subtrees of those nodes that extend S with vertices in B, so they are not removed from g (i.e., not reduction) but just skipped for the level of branching below S. Consider Figure 7 again: assume that $R' = \{d, c\}$ for $S = \{\}$, then even if we skip the subtrees $T_{\{d\}}$ and $T_{\{c\}}$, c and d can still appear in the subtrees $T_{\{b\}}$ and $T_{\{a\}}$.

We use the S-based upper bounding technique to construct R' (and hence $B = R - R'$). As Figure 8 illustrates, our goal is to find R' as a subset of R that is as large as possible so that $UB_S(R') \leq |P|$. Let us view the computation of $UB_S(R')$ as the following process: initially, $UB_S(R')$ is set as $|S|$, and we then obtain $\langle I_i, ub_i \rangle$ from R (which removes elements of I_i from R) one at a time, and add ub_i to $UB_S(R')$. Since we require $UB_S(R') \leq |P|$, we have a quota $\beta = |P| - |S|$ for the summation of ub_i values when we choose as many partitions I_i into R' as possible. Whenever a partition I_i is chosen, we deduct ub_i from β so that β is always the remaining quota. We keep obtaining $\langle I_i, ub_i \rangle$ as long as $\beta \geq ub_i$ (during S-based partition selection as shown in Figure 6, we also do not consider those candidates π_i with $ub_i > \beta$). Finally, as Figure 6 illustrates, assume that $ub_{m+1} > \beta$ for the next partition I_{m+1} (if exists), we then stop obtaining this and future partitions (with potentially lower dise scores), but rather take β more vertices from R (if exists) to prune more branches by letting $UB_S(R')$ reach the allowed value $|P|$. We denote the above process to compute B ($R' = R - B$) as

$$B \leftarrow \text{partition}(S, R, g),$$

and please see [4] for the detailed pseudocode.

Operating on the horizontally flipped set-enumeration tree as shown in Figure 7 allows additional branch pruning as follows: assume that we have processed the top-level branches $T_{\{v_1\}}, T_{\{v_2\}}, \ldots, T_{\{v_{i-1}\}}$ and the current largest k-plex is P, then we can find a k-plex with at most $(|P| + 1)$ vertices in branch $T_{\{v_i\}}$; so as soon as we find such a k-plex, we can skip the rest of $T_{\{v_i\}}$ (where we can find at most ties) and move on to $T_{\{v_{i+1}\}}$. To see this, consider Figure 7 again, and assume that $T_{\{d\}}$, $T_{\{c\}}$ and $T_{\{b\}}$ have been processed so that P is an MkP found over $\{b, c, d\}$. Then in $T_{\{a\}}$, we show that we cannot find a k-plex P' of size more than $|P| + 1$ by contradiction: assume that $|P'| > |P| + 1$, then since a is the only additional vertex beyond $\{b, c, d\}$, we must have $a \in P'$ (otherwise, P is not an MkP over $\{b, c, d\}$); by the hereditary property of k-plex, $P' - \{a\} \subseteq \{b, c, d\}$ is also a k-plex but has size more than $|P|$, which contradicts with the fact that P is an MkP over $\{b, c, d\}$.

Based on this idea, Algorithm 4 shows the partition-based branch-and-bound algorithm to be called by Algorithm 2 Line 10 with σ_{seed} = *true*. Here, σ_{seed} indicates if the function is called by the top level, and within each branch under a top-level node, the recursive call passes *false* to σ_{seed} as Line 11 shows. Thus, when computation just enters a top-level branch $T_{\{v_i\}}$, Line 8 initializes the global flag σ_{up} to *false* to indicate that a k-plex of size $(|P| + 1)$ has not been found in $T_{\{v_i\}}$ yet. Line 11 then recursively extends $S \cup \{v_i\}$ for a vertex $v_i \in B$, and Line 12 checks if the recursive call has increased $|P|$ beyond its old value recorded in Line 10. If so, a larger k-plex is found (and must have size $|P| + 1$ based on the previous discussion), so we terminate the exploration of $T_{\{v_i\}}$ by directly returning in Line 9 along the backtracking path all the way to the top level, which will then proceed the exploration to $T_{\{v_{i+1}\}}$.

Note from Lines 6–7 that we only branch on vertices in B, and we check vertices from $v_{|B|}$ to v_1 in the reverse degeneracy ordering, so that dense regions are examined first in hope that a larger P can be identified early to prune a lot of unpromising branches.

For U-MkP, since reduce_and_prune(.) in Line 3 already calls partition(.) (recall Line 7 of Algorithm 3), Line 4 actually directly gets the computed B (e.g., passed to reduce_and_prune(.) as a reference to collect result) directly rather than run partition(.) again. Since kPEX does not run Line 7 in Algorithm 3, when we implement **kPEX-S** on kPEX, Line 4 runs the actual partition(.) to obtain B.

Pivot-Based Branching. The work of [4] finds that while S-based branching is the most efficient for small k values, it is not competitive to a pivot-based binary branching method when k becomes larger. This is mainly because each $ub_i = \min\{|\pi_i|, \delta_S(v_i)\}$ becomes loose when k is large (since $\delta_S(v_i) = k - |\overline{N_S}(v_i)|$), reducing the pruning power of BB_part(.) (i.e., Algorithm 4).

The pivot-based branching follows the binary branching scheme in Lines 3–5 in Algorithm 1, but it selects a more constraining pivot v at Line 3 for branching, which tends to reduce the remaining candidates in R the most. This is based on the most-constraining-variable heuristic for a constraint satisfaction problem (CSP) which applies to our MkP problem. This branching method is reverse-engineered by [4] from the code of Maple [26], which somehow implements something totally different from what their paper described. It is found to be orders of magnitude faster than other branching methods when k is large, so we implement it on top of kPEX to obtain **kPEX-P**. Due to page limit, we refer readers to Section 4.3 of [4] for the detailed algorithm of pivot-based branching.

3.3 Alternated Reduction-and-Bound

We now explain AltRB — the core reduction technique of kPEX [15] that is called in Line 3 of Algorithm 3 along with other reduction rules. Since the original U-MkP program does not run AltRB, we implement AltRB on top of U-MkP to obtain a new variant **U-MkP$^+$**.

Bounds. Let us partition R into two disjoint sets R_l and R_r as Figure 9 shows. For a k-plex P^* to find from the branch T_S, we have

$$P^* = S + (R_l \cap P^*) + (R_r \cap P^*).$$

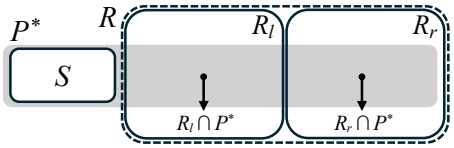


Figure 9: Relation of Lower and Upper Bounds of R_l and R_r

Since we aim to find P^* larger than the current P, we require

$$|S| + \left|R_l \cap P^*\right| + \left|R_r \cap P^*\right| \geq |P| + 1. \tag{4}$$

Let us define UB_l (resp. UB_r) to be the upper bound of $|R_l \cap P^*|$ (resp. $|R_r \cap P^*|$), then we can define a lower bound of $|R_r \cap P^*|$:

$$\begin{aligned} LB_r &\triangleq |P| + 1 - |S| - UB_l \\ &\leq |P| + 1 - |S| - \left|R_l \cap P^*\right| = \left|R_r \cap P^*\right|, \end{aligned} \tag{5}$$

where the last step is due to Eq (4). We can see that if we can reduce (i.e., improve) UB_l, we will also increase (i.e., improve) LB_r.

Symmetrically, we can define a lower bound of $|R_l \cap P^*|$:

$$LB_l \triangleq |P| + 1 - |S| - UB_r. \tag{6}$$

We can see that if we can improve UB_r, we will also improve LB_l.

Upper Bound Computation. Let us assume S is also partitioned into two parts S_l and S_r. Recall from Eq (2) on Page 6 that given $\langle S, R\rangle$, the upper bound on the number of elements in R that can be included into a k-plex extended from S is given by $|\pi_0| + \sum_{i=1}^{|S|} \min\{|\pi_i|, \delta_S(v_i)\}$. We now show how it can be applied directly to $\langle S_l, R_l\rangle$.

Specifically, let us partition the vertices of R_l based on the vertices in S_l as follows: (1) the common neighbors of all vertices in S_l as $\pi_{l,0}$, (2) for each $v_i \in S_l$, obtain $\pi_{l,i}$ as all the remaining non-neighbors of v_i in R. Then we can compute UB_l as follows:

$$UB_l = |\pi_{l,0}| + \sum_{i=1}^{|S_l|} \min\{|\pi_{l,i}|, \delta_{S_l}(v_i)\}. \tag{7}$$

Similar to Eq (3), the vertices $v_i \in S_l$ are selected greedily based on distribution efficiency (dise) to minimize UB_l.

We denote this computation as $UB_l \leftarrow \texttt{ComputeUB}(S_l, R_l)$. Symmetrically, for $\langle S_r, R_r\rangle$, we compute $UB_r \leftarrow \texttt{ComputeUB}(S_r, R_r)$.

Bound-Driven Reduction Rules. Two rules are considered:

R1. Given a branch $\langle S, R\rangle$ with LB_l and LB_r,
- (a) for a vertex v in R_l, we remove v from R if $|N_{S\cup R_l}(v)| < LB_l + |S| - k$ or $|N_{S\cup R_r}(v)| < LB_r + |S| - k + 1$; and
- (b) for a vertex v in R_r, we remove v from R if $|N_{S\cup R_l}(v)| < LB_l + |S| - k + 1$ or $|N_{S\cup R_r}(v)| < LB_r + |S| - k$.

R2. Given a branch $\langle S, R\rangle$ with UB_l and UB_r,
- (a) if $UB_l + UB_r + |S| = |P| + 1$ and $UB_l = |R_l|$, we move all vertices in R_l from R to S if $G[S \cup R_l]$ is a k-plex; otherwise, i.e., it is not a k-plex, we terminate the branch (S, R);
- (b) if $UB_l + UB_r + |S| = |P| + 1$ and $UB_r = |R_r|$, we move all vertices in R_r from R to S if $G[S \cup R_r]$ is a k-plex; otherwise, i.e., it is not a k-plex, we terminate the branch (S, R).

To see R1, consider a vertex $v \in R_l$ with $|N_{S\cup R_l}(v)| < LB_l + |S| - k$, then we can show that if $v \in P^*$, v will miss more than k edges, violating the requirement that P^* is a k-plex:

$$\begin{aligned} |N_{P^*}(v)| &= |N_{P^*\cap(S\cup R_l)}(v)| + |N_{P^*\cap R_r}(v)| \\ &\leq |N_{S\cup R_l}(v)| + |P^* \cap R_r| \\ &< LB_l + |S| - k + |P^* \cap R_r| \ \text{ (using the condition)} \\ &\leq |P^* \cap R_l| + |S| - k + |P^* \cap R_r| \ \leq \ |P^*| - k \end{aligned}$$

The other conditions of R1 can be similarly proved.

To see R2, consider condition $UB_l + UB_r + |S| = |P| + 1$, which when combined with Eq (4), implies that upper bounds UB_l and UB_r are now tight. So for (a), if $UB_l = |R_l|$, then all vertices in R_l is in P^* so should be added to S. If $G[S \cup R_l]$ is already not a k-plex, then $P^* \supseteq S \cup R_l$ cannot be a k-plex due to the hereditary property and the whole branch is pruned. The proof for (b) is symmetric.

Alternated Reduction-and-Bound. When lower bounds are improved, R1 may further shrink R; and when upper bounds are improved, R2 may move vertices from R to S or prune the entire branch. This gives 4-step alternated reduction-and-bound operations:

(1) **Bound on R_l:** Compute $UB_l \leftarrow \texttt{ComputeUB}(S_l, R_l)$.
(2) **Reduction on R_r:** Improve LB_r using Eq (5); refine R_r using rules R1 and R2.
(3) **Bound on R_r:** Compute $UB_r \leftarrow \texttt{ComputeUB}(S_r, R_r)$.
(4) **Reduction on R_l:** Improve LB_l using Eq (6); refine R_l using rules R1 and R2.

Initially, we set $UB_l \leftarrow |R_l|$ and $LB_l \leftarrow 0$, and the above AltRB operations are then repeated until UB_l does not change any more. kPEX also proposes a greedy initialization method for S_l, S_r, R_l and R_r to maximize the pruning effect of R1 and R2, based on the dise of vertices in S. Please refer to Algorithm 3 of [15] for the details.

Finally, the reduced R by AltRB is computed as $R_l \cup R_r$, and an upper bound on the size of k-plex that can be found in T_S is computed as $S + UB_l + UB_r$. If this upper bound is smaller than $|P| + 1$, then the entire branch T_S is pruned.

4 Set Representations

Another difference of U-MkP [4] and kPEX [15] lies in the data structure used for vertex set representation. U-MkP uses a sparse representation that can be reused and incrementally updated during recursive subgraph exploration, while kPEX uses a dense bitmap encoding that is highly efficient for set operations but needs to be created in each recursion body. Both data structures support constant-time vertex insertion, deletion and lookup. Note that since kPEX-S and kPEX-P are implemented on kPEX, they use the bitmap encoding. Similarly, U-MkP$^+$ uses the sparse set representation.

Sparse Representation by Dual Arrays. U-MkP uses the dual-array data structure for set maintenance. Recall that each of our branch-and-bound (BB) algorithm variants (i.e., Algorithms 1 and 4) expands $\langle S, R\rangle$ by recursively calling BB itself over $\langle S', R'\rangle$ where $S' = S \cup \{v_i\}$ is the extended set, and R' is a pruned version of R.

Instead of creating new sets S' and R' as the inputs into each recursive call of BB, we propose to reuse the same containers for S and R throughout the recursive execution for efficiency. The containers are preallocated as length-$|V|$ arrays at the beginning and reused for content initialization and incremental updates.

In particular, we maintain a single containers for both S and R (let us call it SR) using the dual-array data structure shown in

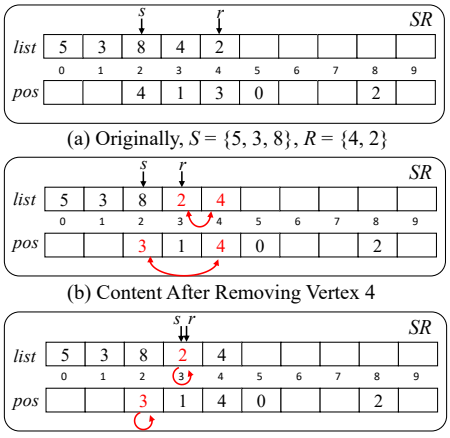


Figure 10: Dual-Array Data Structure for Set Maintenance

Figure 10. Specifically, two arrays with capacity $|V|$ are maintained: (1) *list*[] which keeps the list of elements for scanning and adding new elements; and (2) *pos*[] which maps vertex ID back to its position in *list*[], to facilitate element search and removal. Note that element addition, search and removal can all be done in $O(1)$ time. In Figure 10, the vertices in the *list* array indexed between $[0, s]$ constitute the S set and those between $(s, r]$ constitute the R set.

As Figures 10(a)–(b) show, when removing vertex 4 from g, we first find its position in *list* as $pos[4] = 3$, and then swap this vertex with the last vertex $list[r]$ and finally decrement r. Note that their position values are also swapped in the array *pos*. In this way, the vertices are not completely erased from the structure and can be restored when returning back from the recursive call.

Similarly, as Figures 10(b)–(c) show, when moving vertex 2 from R into S, we first find its position in *list* as $pos[2] = 3$, and then swap this vertex with the first vertex of R, $list[s+1]$, and finally increment the value of s. Note that their position values are also swapped in the array *pos*. In this way, we can incrementally populate S and R for calling BB(.) with minimal cost.

As for Algorithm 4, since R is now split into two sets R' and B, we use a variant of our dual-array data structure, this time with 3 position pointers with b placed between s and r: $[0, s]$ is for S, $(s, b]$ is for R', and $(b, r]$ is for B. The corresponding $O(1)$-time operations for moving vertices around the sets can be similarly derived.

Finally, recall that the S-based branching requires an auxiliary set $\Pi = \cup_{v_i \in S} \pi_i$ that keeps the (potentially overlapping) non-neighbor sets π_i of all $v_i \in S$, used by Line 4 of Algorithm 4 (BB_part(.)) where partition(.) computes the branching set B by first initializing $\Pi = \bigcup_{v \in S} \pi_v$ where for each $v \in S$, $\pi_v = R - N(v)$. Since BB_part(.) is a recursive function, before the top-level Algorithm 2 calls BB_part(.) at Line 10 to process branch $T_{\{v_i\}}$ on $g_i = (V_i, E_i)$ (which is v_i's 2-hop neighborhood graph), we preallocate an array of $|V_i|$ dual-array buffers. Whenever partition(.) is called in this branch, we simply reuse the allocated buffers to initialize Π instead of creating new space. Since locating and removal of a vertex from π_i takes $O(1)$ time, it is efficient to support the removal of vertices of a chosen partition I from each $\pi_i \in \Pi$ as needed by the S-based branching.

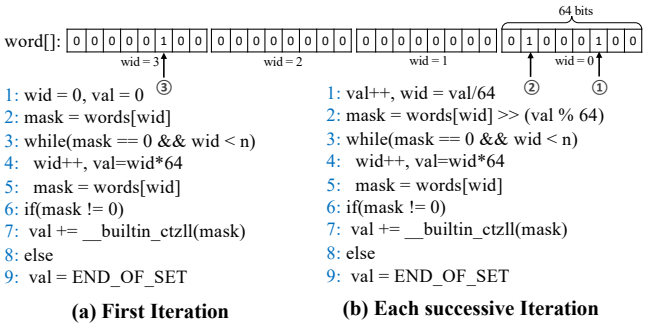


Figure 11: Bitmap Set Representation

Dense Representation by Bitmaps. kPEX [15] adopts a totally different design to use a dense bitmap for each vertex set, which is created for each recursion body. As a tradeoff, set operations are implemented as efficient bitwise operations, which is critical to performance since the pruning rules require intensive set intersection operations, and other operations such as set union (e.g., to move R_l or R_r into S) and difference (e.g., removal of vertices of a chosen partition I from each $\pi_i \in \Pi$) are also frequently used.

Notably, scanning the elements in a set of n elements stored with a bitmap can still be done in $O(n + w)$ time where w is the number of 64-bit words used. This is thanks to the built-in function __builtin_ctzll(mask) that counts the number of trailing zeros in the binary representation of a 64-bit unsigned integer in $O(1)$ time. Since the processing in each branch $T_{\{v_i\}}$ is on $g_i = (V_i, E_i)$, $w = \lceil |V_i|/64 \rceil$ is usually small. Figure 11(a) shows how to get the first element, where `wid` is the word ID and `val` is the bit position of the current element in the entire multi-word bitmap. Lines 1–2 initializes the checking from the first bit of the first word, and Lines 3-5 then skip all-zero words that do not contain elements. If a non-zero word is located (Line 6), we move `val` to the position of the lowest one-bit to return (Line 7), while otherwise, Lines 8–9 indicates that there is no more element in the set. For ①, `val` = 2 will be returned as the first element (__builtin_ctzll(.) returns 2 since there are 2 trailing zeros). Figure 11(b) shows how to get each successive element, where `val++` in Line 1 move the position past the previous element, Line 2 remove the past bits by right shift, and Line 7 counts the trailing bits (i.e., the gap of all-zero bits from the previous one-bit) to be incremented on top of `val` to get the next element position. For ②, Line 1 increments `val` from 2 to 3, Line 2 right-shifts 3 bits for the first word, and Line 7 increments the 3 trailing zeros on top of `val` to return element 6. For ③, Lines 3-4 will skip the second and third all-zero words, with `val` updated as `wid` $*$ 64 = 192; then Line 7 adds 2 to `val` to return element 194.

5 Machine Learning Methods and Experiments

We now present our learning-based method for selecting the fastest MkP algorithm from 5 competitive algorithm candidates: **U-MkP**, **U-MkP$^+$**, **kPEX**, **kPEX-S** and **kPEX-P**. Among them, U-MkP$^+$ adds AltRB to U-MkP, and both adopt the dual-array data structure for sets. kPEX-S and kPEX-P replaces the binary branching in kPEX with S-based and pivot-based branching, respectively, and all three of them adopt the bitmap set representation. All programs are implemented in C++ and compiled using GCC 12.3.0 with -O3 optimization. All experiments are conducted on a server with Intel(R)

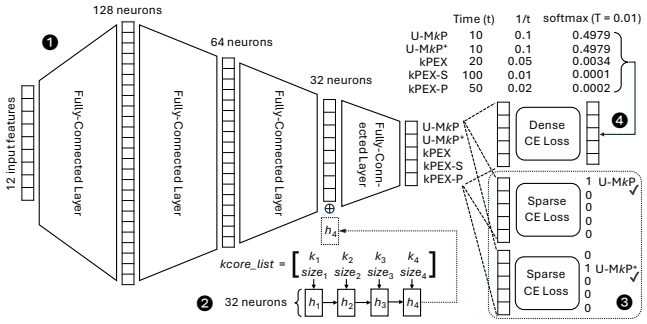


Figure 12: Network Architecture and Tie-Aware Supervision

Xeon(R) Gold 6248R CPU @ 3.00GHz with 48 cores and 754GB of RAM (though our programs never consume more than 10GB).

Dataset. As we have introduced in Section 1, we collect a total of 2741 graphs from SNAP [3], KONECT [1] and Network Repository [2]. Since we need to run MkP algorithms on so many graphs for different values of k, we set a timeout threshold so that if an algorithm runs for more than **1800 seconds**, we treat it as timing out and kill it to make the time of data collection manageable.

During the preprocessing step, the heuristic algorithm can directly find MkP on 1635 graphs. Also, 791 graphs always lead to timeout for any algorithm and any tested k, while 198 graphs entered the subgraph exploration stage but the result is computed in less than a second. Since algorithm selection does not matter much on these graphs, we exclude them. This leads to 117 graphs remaining in our curated dataset that are interesting for our machine learning study. For k, we tested 11 values: 2, 3, 4, 5, 6, 7, 8, 9, 10, 15 and 20. Therefore, we have $117 \times 11 = 1287$ data samples for machine learning, but for some combinations of graph and k, all 5 algorithms run out of time (OOT, i.e., cannot complete in 1800 seconds), so these samples are excluded leaving only 760 samples in our final dataset. Note that this is a small (useful) data challenge for our problem, even though we have initially tries 2741 graphs from 3 popular sources. Therefore, we only explore conventional models and simple deep learning models to avoid overfitting.

Graph features are obtained from the shrunk graph G by CTCP rather than the original graph, since subgraph exploration is conducted over the shrunk graph G. Four features are extracted from G: numbers of vertices and edges, and average and maximum degree.

Recall that a peeling algorithm is conducted by the heuristic algorithm during preprocessing, and CTCP prunes all vertices v with $core(v) < \tau_v = (|P| + 1 - k)$. Therefore, G contains k-cores for $k = \tau_v, \tau_v + 1, \ldots, D(G)$. This gives a sequence feature `kcore_list` where each element is a pair (k, size of k-core), as well as 7 numeric features on k-core statistics such as the length of `kcore_list`, the minimum and maximum k, and the average, maximum, minimum and standard deviation of k-core size in `kcore_list`.

We also treat k as a feature, together with the graph features, we obtain 12 numerical features and one sequential feature `kcore_list`. Note that by incorporating the k-core-related features, we encode richer structural information of a graph to enhance the learning.

Models. We adopt conventional classifier models including random forest (RF), XGBoost (XGB) [11], support vector machine (SVM), logistic regression with no (denoted by LR) or L1 or L2 regularization, or both (Elastic Net, abbr. EN). For deep learning models, we use a feedforward neural network (FNN) and an LSTM-enhanced one.

Here, LSTM is used to embed the sequential feature `kcore_list`, while all the other models only use the 12 numerical features. The input features are z-score normalized for all models except for RF and XGB. All hyperparameters and architectures have been tuned and we report the best model in each category.

For conventional models, the best RF model uses 400 trees and 18 for maximum depth; the best XGB model uses 150 trees and 15 for maximum depth; the best SVM models (both RBF and linear kernels) use 0.1 as regularization parameter; the best LR models use 10^{-3} as regularization parameter, and the best elastic net uses 7×10^{-4} and 3×10^{-4} for L1 and L2 regularization parameters.

For neural networks, the best FNN uses three hidden layers with 128, 64 and 32 neurons, respectively, as shown in Figure 12-❶. Additionally, the best LSTM-enhanced FNN (simply called as LSTM from now on) adds an LSTM branch to extract features from `kcore_list` where the hidden state uses 32 neurons; as Figure 12-❷ shows, the final hidden state is then concatenated with FNN features at the output of the second dense layer to form a 64-dimensional vector, which is then passed to the last dense layer.

Table 2: Examples of Samples with Running-Time Ties

Dataset	k	U-MkP	U-MkP$^+$	kPEX	kPEX-S	kPEX-P
edit-jawiki	3	70.7	66.1	**52.1**	52.6	**52.1**
link-dynamic-itwiki	3	91.2	80.2	**32.7**	**32.7**	32.8
channel-500x100x100-b050	5	**20.9**	**20.9**	49.9	49.4	48.8
orkut-links	15	64.8	64.0	**15.2**	74.0	**15.2**
wikipedia_link_en	9	30.0	17.9	**15.2**	17.3	**15.2**
soc-orkut	8	50.5	56.0	**14.7**	OOT	**14.7**
link-dynamic-dewiki	3	18.9	20.7	11.2	**10.7**	**10.7**
link-dynamic-dewiki	8	45.0	53.4	**10.3**	10.4	**10.3**
soc-digg	10	43.7	18.6	**9.3**	13.5	**9.3**
edit-ptwiki	10	138.2	202.1	**8.6**	1407.3	**8.6**

Here, LSTM is used to embed the sequential feature `kcore_list`, while all the other models only use the 12 numerical features. The input features are z-score normalized for all models except for RF and XGB. All hyperparameters and architectures have been tuned and we report the best model in each category.

For conventional models, the best RF model uses 400 trees and 18 for maximum depth; the best XGB model uses 150 trees and 15 for maximum depth; the best SVM models (both RBF and linear kernels) use 0.1 as regularization parameter; the best LR models use 10^{-3} as regularization parameter, and the best elastic net uses 7×10^{-4} and 3×10^{-4} for L1 and L2 regularization parameters.

For neural networks, the best FNN uses three hidden layers with 128, 64 and 32 neurons, respectively, as shown in Figure 12-❶. Additionally, the best LSTM-enhanced FNN (simply called as LSTM from now on) adds an LSTM branch to extract features from `kcore_list` where the hidden state uses 32 neurons; as Figure 12-❷ shows, the final hidden state is then concatenated with FNN features at the output of the second dense layer to form a 64-dimensional vector, which is then passed to the last dense layer.

Tie Handling. We observe that 187 samples (out of the totally 760) exhibit ties between two or more algorithms that achieve the fastest execution time (denoted by t_{min}), with 10 examples shown in Table 2. For example, on *link-dynamic-dewiki* ($k = 3$), both kPEX-S and kPEX-P achieve 10.7 seconds, so we should count a prediction as correct when it is either kPEX-S or kPEX-P. In fact, we can even include a tie tolerance parameter α, so that algorithms running within $(1+\alpha) \cdot t_{min}$ are also considered a prediction hit. For example, when $\alpha = 10\%$, we can treat all of {kPEX, kPEX-S, kPEX-P} as correct predictions for previous example. With a tolerance parameter $\alpha = 10\%$, 524 samples exhibit ties (69.08% of all samples).

With such a prevalence of ties, we should properly handle ties during both training and evaluation, rather than simply choose one algorithm among the ties as the label. We explored two solutions. The first one is to use *(dense) probabilistic categorical cross-entropy loss* for supervision during training, which applies to logistic regression and neural network models. Figure 12-❹ illustrates this idea, where our 5 candidate algorithms runs for 10, 10, 20, 100 and 50 seconds, respectively. Clearly, the first two algorithms are tied and we expect the probability vector for supervision by cross-entropy (CE) loss to be close to $[0.5, 0.5, 0, 0, 0]$, which can be achieved by computing softmax over the inverse of the time $e^{1/(t_i \cdot T)} / \sum_j e^{1/(t_j \cdot T)}$ for the ith algorithm, where T is the temperature parameter.

However, we find this approach to give a low prediction performance. An alternative approach that works well is a preprocessing

Table 3: Examples of Samples with Different Importance

Dataset	k	U-MkP	U-MkP$^+$	kPEX	kPEX-S	kPEX-P
johnson8-4-4	15	24.0	0.1	7.7	32.8	9.5
cit-Patents	6	504.8	214.5	6.0	0.6	11.0
cit-Patents	3	2.5	2.4	109.5	1.2	69.2
link-dynamic-frwiki	7	OOT	OOT	91.1	3.6	105.2
san200-0-9-1	9	5.2	6.1	0.6	OOT	1.2
soc-flixster	20	0.2	0.2	0.2	0.2	0.2
socfb-B-anon	10	6.6	7.5	3.7	4.0	3.8
wikipedia_link_es	15	7.1	7.1	6.7	6.7	7.1
web-ClueWeb09	2	49.7	41.8	44.6	52.0	57.7
livejournal-groupmemberships	2	259.7	269.1	219.7	219.1	226.1

method (before training) called *tie-aware weight splitting*. Specifically, given a training sample $\mathbf{x}$ with instance weight $w(\mathbf{x})$ (to be explained next), if k algorithms are tied to be all the fastest (or with a tolerance parameter β, e.g., 10% slower than the fastest), then we create k instances of this sample with each instance replicating $\mathbf{x}$'s input features but using a different algorithm label. Each instance has weight $w(\mathbf{x})/k$ so that the weights of all instances sum up to be $w(\mathbf{x})$. Figure 12-❸ shows an example where both U-MkP and U-MkP$^+$ are tied to be the fastest, creating two samples into the training set, each with a weight equal to half of the sample weight.

For tie-aware weight splitting, we notice that using tie tolerance to define the ties would reduce the performance of the trained model. For example, when $\alpha = 0, 0.1, 0.2, 0.5$, the accuracies of RF are 93.68%, 72.76%, 63.68%, 56.18%, respectively, and this performance drop with increasing α is observed in all our tested models. This shows that we should provide strong supervision signals to train a model, where even a small running time difference between two algorithms matters. Therefore, we always use $\alpha = 0$ (i.e., disable tie tolerance) when conducting tie-aware weight splitting.

Sample Weighting. Not all samples are equally important in the context of algorithm selection. In Table 3, the first 5 samples are important since the running time difference among different algorithms are drastically different, while the last 5 samples are less important as the running time difference is not prominent. We identify two important factors on sample weight: (1) the running time of the fastest algorithm t_{min}, and (2) the running time ratio of the fastest v.s. the slowest algorithms t_{max}/t_{min}. We use Table 3 to illustrate why t_{min} is important: intuitively, *soc-flixster* is less important than *wikipedia_link-es*, which is in term less important than *livejournal-groupmemberships*, even though all algorithms have similar running time. Also, even though *socfb-B-anon* has $t_{max}/t_{min} > 2$, we do not feel a wrong prediction critical since the running time of all algorithms are very short anyway.

In this work, we compute sample weight as follows

$$w(\mathbf{x}) = t_{min} \cdot \log\left(\frac{t_{max}}{t_{min} + \epsilon}\right),$$

where $\epsilon = 0.001$ is to avoid division by zero. Note that if $t_{min} = 0$ or if $t_{max} = t_{min}$, then $w(\mathbf{x}) = 0$ since the sample is not important.

When t_{max} is OOT, we need to give it a large value ($\geq$ 1800 seconds) as the OOT penalty for weight computation, and in this work, we simply use 1800 seconds. Also, alternative time-discrepancy-based ways to define $w(\mathbf{x})$ is possible, however, we find that our current setting works well so we do not explore other alternatives. For models from existing libraries, we simply provide the weights as an input argument (e.g., the `sample_weight` argument of `fit(.)`

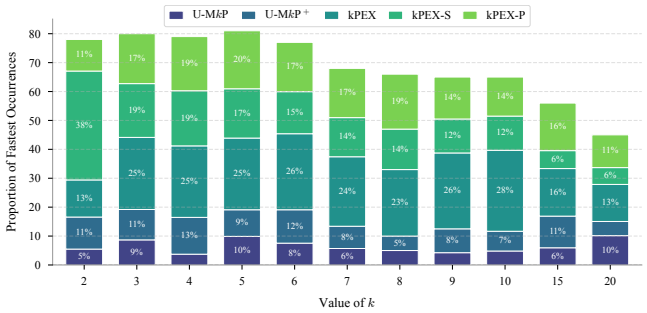


Figure 13: Data Statistics on Algorithm Labels

in scikit-learn), while for models written in PyTorch, we multiply the loss of each sample with its weight for backpropagation.

Metrics. To account for running-time ties, we evaluate predictions using a tie-tolerant methodology. For a given sample, let GT_β denote the set of ground-truth fastest algorithms under a tolerance parameter $\beta \geq 0$, where an algorithm is included in GT_β if it runs for at most $(1+\beta)\cdot t_{\min}$. A prediction is deemed correct if the model selects any algorithm in GT_β. Accordingly, we define: **(1) accuracy@β:** fraction of samples where the predicted algorithm is in GT_β; and for per-class evaluation, **(2) precision@β:** among samples that are predicted to use algorithm $\mathcal{A}$, the proportion where $\mathcal{A} \in GT_\beta$; **(3) recall@β:** among samples where $\mathcal{A} \in GT_\beta$, the proportion whose predictions $\mathcal{A}' \in GT_\beta$. For $\beta = 0$, this reduces to exact matching (no tolerance), while larger β reflects real-world scenarios where near-optimal algorithms are acceptable. We report these metrics across $\beta \in \{0\%, 10\%, 20\%, 50\%\}$.

Dataset Statistics. Figure 13 shows the distribution of algorithm labels for all samples under each value of k, where if k algorithms are tied for a sample, then we add $1/k$ to each label. We use 10% tolerance when determining ties, and we can see that more samples are missing for larger k since those samples run OOT for all algorithms. It is clear that for all values of k, every algorithm occurs as the fastest on a certain fraction of samples. Also, kPEX variants win more often than U-MkP variants, with kPEX itself winning in more cases than any other variant other than when $k = 2$, in which case kPEX-S clearly wins more often. There is no algorithm that wins all the time, showing the demand of algorithm selection methods.

We randomly split the dataset with 80% in the training set and 20% in the test set. The training samples are then further tie-aware weight split (as illustrated in Figure 12-❸). We use stratified sampling on both the algorithm label and k to ensure that the training and test sets have the same distribution of labels and values of k.

Algorithm Prediction Performance. Table 4 shows the performance of all models when using tie tolerance parametes $\beta = 0, 0.1, 0.2$ and 0.5 for evaluation (i.e., a prediction is considered correct if it is within $(1+\beta)\cdot t_{min}$), where algorithms with an asterisk are the versions using dense probabilistic categorical cross-entropy (CE) loss for supervision rather than using tie-aware weight splitting. We can clearly see that the dense CE loss supervision actually hurts the performance, when compared with versions that use tie-aware weight splitting. We have tuned the temperature parameter for the dense CE loss and find that 1 works the best, so the reported results are with temperature as 1. We can see that in our setting, what really helps is a clean label (i.e., one-hot encoding) to provide

Table 4: Performance of Different Machine Learning Models on Fastest Algorithm Prediction

	Algorithm	RF	XGB	SVC (RBF)	SVC (Linear)	LR	L1	L2	EN	LR*	L1*	L2*	EN*	FNN	FNN*	LSTM	LSTM*
precision@0	accuracy@0	93.7	93.0	56.1	57.4	55.0	55.0	54.0	54.3	34.2	32.4	33.7	34.2	64.2	33.3	59.7	33.3
	U-MkP	94.0	92.7	36.8	52.4	42.9	44.7	42.9	46.4	28.6	6.7	14.3	8.3	58.5	3.9	68.2	15.4
	U-MkP+	90.1	89.4	61.4	49.0	55.8	60.0	53.7	54.6	35.3	22.2	29.7	40.0	63.6	60.4	56.7	0.0
	kPEX	94.7	93.6	62.3	64.8	57.0	56.6	55.6	56.1	25.0	0.0	0.0	0.0	67.6	22.2	62.5	0.0
	kPEX-S	92.4	95.6	49.0	46.3	47.4	46.7	48.2	47.7	34.9	34.9	35.2	35.3	60.1	38.8	65.1	34.3
	kPEX-P	95.5	91.1	0.0	66.7	80.0	50.0	0.0	50.0	21.4	13.6	19.4	19.1	60.0	11.0	51.6	0.0
recall@0	U-MkP	93.0	92.0	53.0	51.0	46.0	48.0	36.0	39.0	24.0	21.0	22.0	21.0	59.0	23.0	45.0	24.0
	U-MkP+	93.1	94.0	56.0	61.2	56.0	54.3	55.2	54.3	29.3	17.2	24.1	27.6	50.9	38.8	68.1	14.7
	kPEX	95.0	95.9	76.3	81.0	90.6	91.2	90.1	90.6	27.6	26.2	26.2	26.2	89.0	26.2	86.5	26.5
	kPEX-S	94.3	96.5	62.9	59.0	46.9	45.7	47.3	47.3	89.8	93.0	92.6	93.4	70.7	81.6	60.6	96.9
	kPEX-P	96.9	92.0	50.4	51.8	53.5	53.1	51.3	52.7	42.5	42.5	43.4	42.9	59.7	46.0	58.0	40.7
precision@0.1	accuracy@0.1	97.9	98.3	72.2	75.0	74.9	74.7	73.2	73.4	46.7	43.3	45.0	45.8	81.3	48.0	79.1	44.6
	U-MkP	97.6	97.1	44.7	66.7	54.8	57.5	47.6	53.6	50.0	13.3	19.1	25.0	81.5	7.8	81.8	25.6
	U-MkP+	95.6	97.9	84.3	71.2	89.6	91.4	82.9	83.1	52.9	38.9	48.7	55.0	85.5	95.8	86.5	0.0
	kPEX	98.2	98.6	78.8	81.6	75.9	75.5	74.5	75.0	60.7	0.0	0.0	0.0	82.6	44.4	79.6	0.0
	kPEX-S	98.4	98.9	65.4	64.4	66.0	66.7	64.3	64.2	45.3	45.4	45.9	45.9	77.1	50.0	76.7	45.6
	kPEX-P	98.3	97.8	0.0	66.7	80.0	50.0	0.0	50.0	57.1	40.9	38.7	47.6	82.0	37.3	73.0	0.0
recall@0.1	U-MkP	96.6	98.6	72.1	74.2	72.1	70.8	61.9	63.3	39.5	27.2	32.7	35.4	74.8	44.9	78.2	26.5
	U-MkP+	95.9	97.1	65.3	70.6	66.5	65.3	59.4	60.6	35.3	21.8	29.4	31.8	71.2	44.1	76.5	17.1
	kPEX	98.6	99.4	82.8	86.1	94.3	94.9	93.9	93.9	47.9	46.0	46.6	46.4	94.1	49.1	92.2	48.5
	kPEX-S	98.8	98.8	78.9	77.7	73.0	72.1	73.6	73.3	93.3	93.0	93.6	93.8	85.6	87.7	80.1	97.1
	kPEX-P	98.2	99.0	77.9	81.5	87.8	88.0	87.1	87.1	54.7	53.0	53.7	53.5	90.2	57.1	89.5	54.9
precision@0.2	accuracy@0.2	98.7	98.8	78.0	80.5	79.7	79.7	77.9	78.3	52.6	49.5	51.2	52.1	85.8	54.6	83.7	50.3
	U-MkP	98.8	98.5	55.3	71.4	61.9	63.8	52.4	57.1	50.0	16.7	19.1	29.2	86.2	7.8	86.4	28.2
	U-MkP+	97.8	98.9	85.7	76.0	90.9	92.9	86.6	87.0	52.9	38.9	48.7	55.0	85.5	95.8	89.4	0.0
	kPEX	99.1	99.3	83.7	86.3	80.9	80.8	79.3	79.8	67.9	0.0	0.0	0.0	87.2	44.4	84.5	0.0
	kPEX-S	98.9	98.9	72.6	71.8	72.2	72.2	69.6	70.6	51.9	51.9	52.5	52.4	82.0	56.6	79.1	51.5
	kPEX-P	98.3	97.8	0.0	100.0	80.0	50.0	0.0	50.0	57.1	50.0	48.4	61.9	88.0	50.0	78.6	0.0
recall@0.2	U-MkP	98.3	98.8	75.1	77.5	75.1	74.0	67.1	68.8	44.5	33.0	37.6	41.0	76.3	49.7	81.5	32.4
	U-MkP+	97.4	97.9	71.1	75.3	70.5	69.5	63.7	65.3	43.2	32.1	38.4	41.1	75.8	49.0	79.5	27.9
	kPEX	99.2	99.6	86.0	89.2	95.1	95.5	94.9	94.9	56.1	54.9	55.5	55.3	95.6	58.7	94.3	56.3
	kPEX-S	99.6	99.5	83.9	83.1	79.2	79.0	79.5	79.5	94.0	94.0	94.3	94.6	90.1	88.3	84.9	97.1
	kPEX-P	99.5	99.8	84.9	88.0	93.7	93.9	93.1	93.3	58.3	57.0	57.6	57.4	94.5	61.5	94.3	58.5
precision@0.5	accuracy@0.5	99.1	99.3	85.1	86.8	86.3	86.5	84.7	84.9	58.4	55.9	56.8	58.0	90.3	60.7	88.7	57.2
	U-MkP	100.0	100.0	76.3	85.7	71.4	74.5	66.7	67.9	64.3	30.0	23.8	41.7	89.2	9.8	90.9	33.3
	U-MkP+	97.8	98.9	92.9	87.5	97.4	97.1	92.7	93.5	56.9	44.4	54.1	62.5	92.7	100.0	94.2	0.0
	kPEX	99.1	99.7	87.9	89.6	86.3	86.4	85.0	85.5	78.6	0.0	0.0	0.0	91.7	55.6	88.2	0.0
	kPEX-S	100	98.9	80.3	80.3	83.5	84.4	81.3	80.7	57.6	57.7	58.0	58.1	86.3	61.6	93.0	58.5
	kPEX-P	98.3	99.3	0.0	100.0	100.0	100.0	0.0	50.0	57.1	72.7	58.1	66.7	92.0	62.7	84.9	0.0
recall@0.5	U-MkP	99.2	99.6	88.8	89.2	89.2	88.8	83.5	84.3	60.2	53.0	56.6	59.0	90.8	63.9	93.2	54.2
	U-MkP+	98.8	99.2	83.0	85.3	83.4	83.8	78.8	79.5	57.9	51.0	54.4	56.8	86.9	61.0	88.8	50.6
	kPEX	99.5	99.8	91.2	93.3	96.7	97.1	96.7	96.6	62.3	62.0	61.8	62.0	96.4	65.4	95.9	63.9
	kPEX-S	99.5	99.8	90.4	89.0	86.7	86.7	87.2	86.9	95.2	94.3	94.0	95.0	93.8	90.1	90.4	97.5
	kPEX-P	99.8	99.8	90.9	92.5	96.2	96.5	96.2	96.0	62.8	62.2	62.1	62.2	96.3	65.9	95.6	64.0

Table 5: Failure Cases for XGB & SVC (RBF Kernel)

Dataset	k	Model	Fastest Time	Predicted Time
hamming6-2	**6**	**SVC (RBF)**	**0.0**	**11.2**
hamming6-2	**6**	**XGB**	**0.0**	**6.7**
link-dynamic-dewiki	2	XGB	13.0	24.7
p-hat500-1	5	XGB	164.8	210.1
soc-digg	**20**	**SVC (RBF)**	**8.1**	**10.1**
soc-digg	**20**	**XGB**	**8.1**	**10.1**
wiki_talk_en	**2**	**SVC (RBF)**	**783.2**	**835.5**
wiki_talk_en	**2**	**XGB**	**783.2**	**OOT**
wikipedia_link_fr	10	XGB	7.0	26.7
wikipedia_link_fr	15	XGB	1.2	8.9
wikipedia_link_it	**4**	**SVC (RBF)**	**3.7**	**4.0**
wikipedia_link_it	**4**	**XGB**	**3.7**	**5.7**
wikipedia_link_sr	**20**	**SVC (RBF)**	**18.0**	**738.1**
wikipedia_link_sr	**20**	**XGB**	**18.0**	**22.6**

strong supervision, and dense CE loss weakens this ground; this is similar to the reason why we set $\alpha = 0$ for tie-aware weight splitting. In summary, the takeaway message from our experiments is that we should stick to tie-aware weight splitting without tie tolerance (i.e., $\alpha = 0$) when training our algorithm prediction models.

Table 4 also shows that the two tree ensemble models perform surprisingly well, both achieving a prediction accuracy above 93% which improves above 99% if we allow 50% time tolerance above the fastest algorithm. However, the other models do not perform well. So the takeaway message is that we should stick to a tree ensemble model such as XGBoost or Random Forest for training our model for predicting the fastest algorithm.

Duel-to-Kill (DTK). While XGB achieves 93% accuracy on our dataset with 760 samples, there are still some failure cases. We thus printed the failure cases and find that the vast majority of them are unimportant ones, such as (1) when both the fastest and the predicted times are below 5 seconds, and (2) the predicted time is within 1.2× the fastest time. After excluding these, we only have 9 failure cases left as shown in Table 5, where critical failures include *wiki_talk_en* where XGB runs OOT, and *link-dynamic-dewiki*, *p-hat500-1* and *wikipedia_link_fr* with significant slowdown.

We propose a runtime technique Duel-to-Kill (DTK), where two algorithms duel by running concurrently, and the first to complete kills the other. This ensures short latency by using an additional CPU core (could otherwise be idle anyway). We pair XGBoost with SVM (RBF Kernel) for DTK, and Table 5 shows the result of inner-joining the above mentioned XGB failure cases with all failure cases from SVM (there are 332 out of 760 samples) on (dataset, k). We can see that SVM can find the fastest model on four datasets that XGB failed (i.e., the black rows for XGB where SVM succeeds).

Among the other XGB failure cases, on *wiki_talk_en*, SVM predicts an algorithm running for 835.5 seconds (close to the fastest time), while XGB runs OOT and will be timely killed. The other XGB failure cases (in color) are not that serious even though SVM does not fix them. For example, SVM runs much slower on *wikipedia_link_sr* but XGB will run for 22.6 seconds and kill it, so the total running time is not too far from the fastest (which is 18 seconds).

6 Conclusion

We implemented five competitive algorithm candidates for finding a maximum k-plex, which are variants of the two best existing algorithms U-MkP and kPEX. Since there is no clear winner, a machine learning solution is explored with time-discrepancy-based sample weighting, and tie-aware weight splitting. Experiments show that XGBoost achieves a prediction accuracy of 93%, and when paired with SVM (RBF kernel), can avoid the few critical prediction failures of using XGBoost alone, achieving ideal algorithm selection.

References

[1] [n. d.]. KONECT. http://konect.cc/networks/.
[2] [n. d.]. Network Repository. https://networkrepository.com.
[3] [n. d.]. SNAP. https://snap.stanford.edu/.
[4] Akhlaque Ahmad, Da Yan, Xiao Chen, Lyuheng Yuan, Qin Zhang, and Saugat Adhikari. 2025. Maximum k-Plex Finding: Choices of Pruning Techniques Matter! *Proc. VLDB Endow.* 14, 9 (2025).
[5] Gary D Bader and Christopher WV Hogue. 2003. An Automated Method for Finding Molecular Complexes in Large Protein Interaction Networks. *BMC bioinformatics* 4, 1 (2003), 2.
[6] Balabhaskar Balasundaram, Sergiy Butenko, and Illya V. Hicks. 2011. Clique Relaxations in Social Network Analysis: The Maximum k-Plex Problem. *Oper. Res.* 59, 1 (2011), 133–142.
[7] Vladimir Batagelj and Matjaz Zaversnik. 2003. An O(m) Algorithm for Cores Decomposition of Networks. *CoRR* cs.DS/0310049 (2003). http://arxiv.org/abs/cs/0310049
[8] Dongbo Bu, Yi Zhao, Lun Cai, Hong Xue, Xiaopeng Zhu, Hongchao Lu, Jingfen Zhang, Shiwei Sun, Lunjiang Ling, Nan Zhang, et al. 2003. Topological Structure Analysis of the Protein-Protein Interaction Network in Budding Yeast. *Nucleic acids research* 31, 9 (2003), 2443–2450.
[9] Lijun Chang, Mouyi Xu, and Darren Strash. 2022. Efficient Maximum k-Plex Computation over Large Sparse Graphs. *Proc. VLDB Endow.* 16, 2 (2022), 127–139.
[10] Lijun Chang and Kai Yao. 2024. Maximum k-Plex Computation: Theory and Practice. *Proc. ACM Manag. Data* 2, 1 (2024), 63:1–63:26.
[11] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *KDD*, Balaji Krishnapuram, Mohak Shah, Alexander J. Smola, Charu C. Aggarwal, Dou Shen, and Rajeev Rastogi (Eds.). ACM, 785–794.
[12] Alessio Conte, Donatella Firmani, Caterina Mordente, Maurizio Patrignani, and Riccardo Torlone. 2017. Fast Enumeration of Large k-Plexes. In *KDD*. ACM, 115–124.
[13] Alessio Conte, Tiziano De Matteis, Daniele De Sensi, Roberto Grossi, Andrea Marino, and Luca Versari. 2018. D2K: Scalable Community Detection in Massive Networks via Small-Diameter k-Plexes. In *KDD*. ACM, 1272–1281.
[14] Jian Gao, Jiejiang Chen, Minghao Yin, Rong Chen, and Yiyuan Wang. 2018. An Exact Algorithm for Maximum k-Plexes in Massive Graphs. In *IJCAI*. ijcai.org, 1449–1455.
[15] Shuohao Gao, Kaiqiang Yu, Shengxin Liu, and Cheng Long. 2024. Maximum k-Plex Search: An Alternated Reduction-and-Bound Method. *Proc. VLDB Endow.* 18, 2 (2024), 363–376.
[16] John Hopcroft, Omar Khan, Brian Kulis, and Bart Selman. 2004. Tracking Evolving Communities in Large Linked Networks. *Proceedings of the National Academy of Sciences* 101, suppl 1 (2004), 5249–5253.
[17] Haiyan Hu, Xifeng Yan, Yu Huang, Jiawei Han, and Xianghong Jasmine Zhou. 2005. Mining Coherent Dense Subgraphs Across Massive Biological Networks for Functional Discovery. *Bioinformatics* 21, suppl_1 (2005), i213–i221.
[18] Hua Jiang, Fusheng Xu, Zhifei Zheng, Bowen Wang, and Wei Zhou. 2023. A Refined Upper Bound and Inprocessing for the Maximum K-plex Problem. In *IJCAI*. ijcai.org, 5613–5621.
[19] Hua Jiang, Dongming Zhu, Zhichao Xie, Shaowen Yao, and Zhang-Hua Fu. 2021. A New Upper Bound Based on Vertex Partitioning for the Maximum K-plex Problem. In *IJCAI*. ijcai.org, 1689–1696.
[20] John M. Lewis and Mihalis Yannakakis. 1980. The Node-Deletion Problem for Hereditary Properties is NP-Complete. *J. Comput. Syst. Sci.* 20, 2 (1980), 219–230.
[21] Junqiu Li, Xingyuan Wang, and Yaozu Cui. 2014. Uncovering the Overlapping Community Structure of Complex Networks by Maximal Cliques. *Physica A: Statistical Mechanics and its Applications* 415 (2014), 398–406.
[22] Jeffrey Pattillo, Nataly Youssef, and Sergiy Butenko. 2013. On Clique Relaxation Models in Network Analysis. *European Journal of Operational Research* 226, 1 (2013), 9–18.
[23] Stephen B Seidman. 1983. Network structure and minimum degree. *Social networks* 5, 3 (1983), 269–287.
[24] Stephen B Seidman and Brian L Foster. 1978. A Graph-Theoretic Generalization of the Clique Concept. *Journal of Mathematical Sociology* 6, 1 (1978), 139–154.
[25] Duygu Ucar, Sitaram Asur, Umit Catalyurek, and Srinivasan Parthasarathy. 2006. Improving Functional Modularity in Protein-Protein Interactions Graphs using Hub-Induced Subgraphs. In *European Conference on Principles of Data Mining and Knowledge Discovery*. Springer, 371–382.
[26] Zhengren Wang, Yi Zhou, Chunyu Luo, and Mingyu Xiao. 2023. A Fast Maximum k-Plex Algorithm Parameterized by the Degeneracy Gap. In *IJCAI*. ijcai.org, 5648–5656.
[27] Zhengren Wang, Yi Zhou, Mingyu Xiao, and Bakhadyr Khoussainov. 2022. Listing Maximal k-Plexes in Large Real-World Graphs. In *WWW*. ACM, 1517–1527.
[28] Mingyu Xiao, Weibo Lin, Yuanshun Dai, and Yifeng Zeng. 2017. A Fast Algorithm to Compute Maximum k-Plexes in Social Network Analysis. In *AAAI*. AAAI Press, 919–925.
[29] Jiongzhi Zheng, Mingming Jin, Yan Jin, and Kun He. 2023. Two New Upper Bounds for the Maximum k-plex Problem. *CoRR* abs/2301.07300 (2023).
[30] Yi Zhou, Shan Hu, Mingyu Xiao, and Zhang-Hua Fu. 2021. Improving Maximum k-plex Solver via Second-Order Reduction and Graph Color Bounding. In *AAAI*. AAAI Press, 12453–12460.